

Elevator Control System Simulation

Using Object-Oriented Programming and State Design Pattern

SUBMITTED BY

Student Name	Silvia Singh
Roll Number	19
Instructor	Dr. Soumadip Biswas
Course	PCCCS495 – Object Oriented Programming
Programme	B.Tech CSE (IoT) – 4th Semester
Submission Date	March 2026

1. Problem Statement

Elevators are one of the most essential systems in modern multi-storey infrastructure — from office towers and hospitals to shopping complexes and academic institutions. While the user experience is deceptively simple — press a button and the elevator arrives — the underlying control mechanism must simultaneously manage multiple floor requests, track the elevator's current position, determine the appropriate direction of travel, and transition seamlessly through distinct operational phases.

Without a well-designed control strategy, an elevator may respond to requests in an arbitrary order, leading to unnecessary back-and-forth movement, excessive waiting times, and inefficient floor coverage. This problem becomes significantly more pronounced during peak hours when the frequency of requests is high.

This project addresses the challenge by simulating a fully functional elevator control system in Java. The simulation accepts user-specified floor requests at runtime, validates them, and processes them in the correct directional order. The state of the elevator — whether it is idle, moving upward, or moving downward — is modelled using the State Design Pattern, where each operational phase is encapsulated in a dedicated class implementing a common State interface.

The system uses a LinkedList-based Queue for managing incoming requests in a first-come, first-served manner. Direction determination is handled dynamically: each dequeued request is compared against the elevator's current floor to decide whether `MovingUpState`, `MovingDownState`, or `IdleState` is appropriate. Validation logic ensures only floors within the permissible range (0–10) are accepted, with invalid inputs rejected gracefully. Floor traversal is simulated step by step, and door operations are printed at each stop to reflect realistic elevator behavior.

The project demonstrates how core OOP principles — abstraction, polymorphism, encapsulation, and exception handling — can be applied cohesively to simulate a real-world control system in a clean, maintainable, and extensible manner.

2. System Design

2.1 UML Class Diagram

The class diagram below illustrates the complete structural relationships between components. The Elevator class serves as the central entity, maintaining the current floor and direction. ElevatorController wraps Elevator and manages a Queue of floor requests, delegating movement logic to the appropriate State implementation. The State interface defines the contract that all concrete states must satisfy. The Request class provides a typed wrapper for floor numbers, available for future integration.

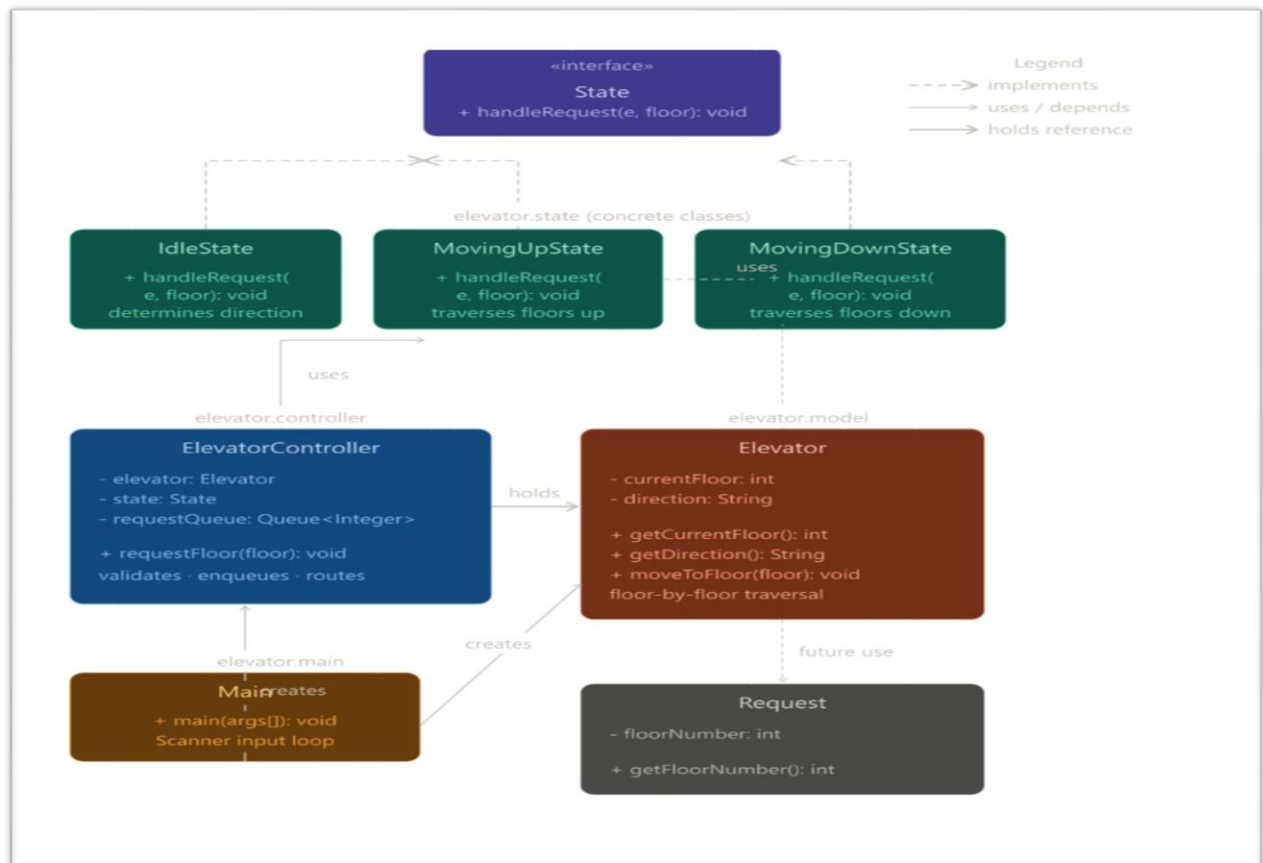


Figure 1: UML Class Diagram — Elevator Control System

2.2 Description of Major Classes

Class / Package	Description
Elevator <code>elevator.model</code>	Central entity. Initialises at floor 0 with direction 'IDLE'. Exposes <code>getCurrentFloor()</code> and <code>moveToFloor()</code> , which traverses floor-by-floor printing each step. Fields <code>currentFloor</code> and <code>direction</code> are private.
ElevatorController <code>elevator.controller</code>	Manages a LinkedList-based Queue of integer floor requests. <code>requestFloor()</code> validates input (0–10), enqueues the floor, then dequeues and processes each request by selecting and invoking the appropriate State.

State <code>elevator.state</code> (interface)	Defines the single-method contract: <code>handleRequest(Elevator, int)</code> . All concrete state classes implement this interface, enabling runtime polymorphism across idle, upward, and downward behaviors.
IdleState <code>elevator.state</code>	Invoked when the requested floor equals the current floor. Prints a status message and determines whether a direction switch will be needed, logging the intended transition.
MovingUpState <code>elevator.state</code>	Sets direction to upward. Traverses floor-by-floor using a while loop from <code>currentFloor</code> to destination. Prints door-open and door-close messages upon arrival.
MovingDownState <code>elevator.state</code>	Mirrors <code>MovingUpState</code> for downward travel. Decrements floor-by-floor until destination is reached, then prints arrival and door operation messages.
Request <code>elevator.model</code>	A model class storing a <code>floorNumber</code> . Provides <code>getFloorNumber()</code> for retrieval. Available for future extension where source-destination pairs may be required.

2.3 Package Structure

The project follows a clean, logical package hierarchy separating concerns across four distinct layers:

```
src/  
  elevator/  
    controller/  
      ElevatorController.java  
    main/  
      Main.java  
    model/  
      Elevator.java  
      Request.java  
    state/  
      State.java           // interface  
      IdleState.java  
      MovingUpState.java  
      MovingDownState.java
```

Each package has a single, well-defined responsibility: model holds data entities; state encapsulates behavior; controller orchestrates logic; and main drives the user interaction loop. This separation ensures future modifications — such as adding an `EmergencyState` — can be made without disturbing the rest of the codebase.

2.4 Interaction Flow

The following sequence describes the runtime interaction between components:

- Main instantiates an `Elevator` (floor 0, direction IDLE) and an `ElevatorController` wrapping it.
- The user enters the number of requests, then inputs each floor number via `Scanner`.
- Each call to `requestFloor()` validates the input — floors outside 0–10 are rejected with a printed message.
- Valid floors are enqueued into the `LinkedList`-based `Queue` inside `ElevatorController`.
- The while loop polls each floor and compares it to the elevator's current floor to select the appropriate `State`: `MovingUpState`, `MovingDownState`, or `IdleState`.
- The selected state's `handleRequest()` is invoked: it traverses floor-by-floor, then prints door-open and door-close messages upon arrival.
- Control returns to the queue loop until all requests are fully processed.

3. OOP Concepts Demonstrated

The table below provides a consolidated summary of every OOP concept applied in this project:

OOP Concept	Where Used	Purpose
Abstraction	State interface	Hides movement logic behind a single <code>handleRequest()</code> contract
Polymorphism	<code>IdleState</code> , <code>MovingUpState</code> , <code>MovingDownState</code>	Same method call behaves differently per state at runtime
Encapsulation	<code>Elevator</code> — <code>currentFloor</code> , <code>direction</code> private	Fields protected via getters; no direct external mutation
Exception Handling	<code>ElevatorController.requestFloor()</code>	Invalid floors (< 0 or > 10) rejected gracefully with a message
Collections	<code>LinkedList</code> <code>Queue</code> in <code>ElevatorController</code>	FIFO request processing; supports multiple back-to-back requests
Interface Inheritance	/ State implementations	All state classes share the State contract via interface implementation

3.1 Abstraction

The `State` interface in `elevator.state` abstracts all movement behavior behind a single method. `ElevatorController` never needs to know which state is currently active — it simply calls `handleRequest()` and the correct logic executes, decoupling the controller from implementation details entirely.

```
// State.java
package elevator.state;
import elevator.model.Elevator;

public interface State {
    void handleRequest(Elevator elevator, int floor);
}
```

This single-method interface is the foundation of the State Design Pattern in this project. Any new operational state — such as `EmergencyState` or `MaintenanceState` — only needs to implement this one method and integrates seamlessly without modifying any existing class, satisfying the Open/Closed Principle.

3.2 Polymorphism

Runtime polymorphism is achieved through the State pattern. When `ElevatorController` invokes `state.handleRequest()`, the JVM dispatches to the correct implementation based on the actual runtime type of the state object assigned for that floor request.

```
// ElevatorController.java - same call, different behavior
if (f > elevator.getCurrentFloor()) {
    state = new MovingUpState();
} else if (f < elevator.getCurrentFloor()) {
    state = new MovingDownState();
}
```

```
} else {  
    state = new IdleState();  
}  
state.handleRequest(elevator, f);    // dispatches to correct class
```

The same line `state.handleRequest(elevator, f)` produces entirely different behavior — upward traversal, downward traversal, or an idle message — depending solely on which concrete `State` object has been assigned. This is the essence of runtime polymorphism: one interface, multiple behaviors.

3.3 Encapsulation

The `Elevator` class keeps its internal fields strictly private. No external class can directly manipulate `currentFloor` or `direction`; they must use the defined public accessor methods, ensuring the object always remains in a consistent, valid state.

```
// Elevator.java  
public class Elevator {  
    private int currentFloor;  
    private String direction;  
  
    public Elevator() {  
        currentFloor = 0;  
        direction = "IDLE";  
    }  
  
    public int getCurrentFloor() { return currentFloor; }  
    public String getDirection() { return direction; }  
}
```

Without encapsulation, any external class could set `currentFloor` to a negative value or assign a nonsensical `direction` string. Private fields with controlled accessors enforce data integrity across the entire system.

3.4 Exception Handling

Floor validation in `requestFloor()` uses a conditional guard to reject any floor outside the permissible range of 0 to 10. Rather than throwing an unchecked exception that could crash the program, it prints a descriptive message and returns gracefully, allowing remaining valid requests in the same session to be processed.

```
// ElevatorController.java  
public void requestFloor(int floor) {  
    if (floor < 0 || floor > 10) {  
        System.out.println("Invalid floor: " + floor);  
        return;  
    }  
    requestQueue.add(floor);  
    // ... process queue  
}
```

This approach ensures robustness: a single bad input does not terminate the session. The controller continues to process all other valid requests — the expected behavior of a production-grade system where one invalid call must never bring the entire system down.

3.5 Collections

ElevatorController uses a LinkedList-backed Queue to store incoming floor requests. This guarantees first-in, first-served (FIFO) processing: the elevator services floors in the exact order they were requested, which is both intuitive and fair from a user perspective.

```
// ElevatorController.java
private Queue<Integer> requestQueue = new LinkedList<>();

requestQueue.add(floor);          // enqueue

while (!requestQueue.isEmpty()) { // dequeue and process
    int f = requestQueue.poll();
    // ... determine state and invoke handleRequest()
}
```

The Queue abstraction also makes the design extensible: replacing LinkedList with a PriorityQueue would immediately enable SCAN-style scheduling without changing any other class. This demonstrates the power of programming to an interface rather than a concrete implementation.

4. Implementation Highlights

Highlight 1 — Queue-Based Request Management

The most significant architectural decision in ElevatorController is the choice of a LinkedList-backed Queue for managing floor requests. This allows the system to accept multiple floor inputs upfront, enqueue them all, and then process them sequentially — mimicking a real elevator control panel that queues button presses.

```
private Queue<Integer> requestQueue = new LinkedList<>();

requestQueue.add(floor);

while (!requestQueue.isEmpty()) {
    int f = requestQueue.poll();
    if (f > elevator.getCurrentFloor()) state = new MovingUpState();
    else if (f < elevator.getCurrentFloor()) state = new MovingDownState();
    else state = new IdleState();
    state.handleRequest(elevator, f);
}
```

Each poll() removes and returns the head of the queue. The direction check immediately after ensures the correct state is selected for every individual request, even as the elevator's currentFloor changes between requests. This makes the system stateful and correct across multi-request sessions.

Highlight 2 — Step-by-Step Floor Traversal

Rather than teleporting the elevator directly to a destination, both MovingUpState and MovingDownState traverse floor-by-floor using a while loop. This ensures currentFloor is updated incrementally, which is critical for correctness when subsequent requests depend on the elevator's position.

```
// MovingUpState.java
```

```
public void handleRequest(Elevator elevator, int floor) {
    System.out.println("Elevator is Moving Up");
    while (elevator.getCurrentFloor() < floor) {
        int nextFloor = elevator.getCurrentFloor() + 1;
        elevator.moveToFloor(nextFloor);
    }
    System.out.println("Reached Floor " + floor);
    System.out.println("Door Opening...");
    System.out.println("Door Closing...");
    System.out.println("Request completed successfully");
}
```

The floor-by-floor approach means `moveToFloor()` is called once per floor, printing a move message each time. This produces a clear, traceable output log that reflects real elevator travel and makes the simulation easy to follow and verify during testing and viva.

Highlight 3 — State Selection Logic in ElevatorController

The state selection block inside `ElevatorController` is the decision engine of the entire system. For every dequeued floor, it evaluates the elevator's current position and assigns the correct `State` object before invoking `handleRequest()`. This clean three-branch conditional keeps controller logic transparent and easy to reason about.

```
// ElevatorController.java - direction decision engine
System.out.println("\n=====");
System.out.println("Request received for floor: " + floor);
System.out.println("=====");

while (!requestQueue.isEmpty()) {
    int f = requestQueue.poll();
    if (f > elevator.getCurrentFloor()) state = new MovingUpState();
    else if (f < elevator.getCurrentFloor()) state = new MovingDownState();
    else state = new IdleState();
    state.handleRequest(elevator, f);
}
```

Importantly, the state is re-evaluated for every floor — not just once at the start of a session. This means the system correctly handles mixed requests (e.g., go up to floor 7, then come down to floor 3) without any additional orchestration logic.

5. Testing & Error Handling

5.1 Test Cases

The following test cases were executed manually by running the simulation with varied inputs and verifying console output against expected behavior:

TC	Input	Expected Output	Result	Status
TC-01	Floor 5 (current: 0)	MovingUpState; floors 1→5 printed; door open/close	Pass	✓ Verified
TC-02	Floor 2 (current: 5)	MovingDownState; floors 4→2 printed; door open/close	Pass	✓ Verified

TC-03	Floor 0 (current: 0)	IdleState; 'Elevator is in Idle State' printed	Pass	✓ Verified
TC-04	Floors 3, 7 (current: 0)	UP to 3 (door), then UP to 7 (door); sequential correct	Pass	✓ Verified
TC-05	Floor -1	'Invalid floor: -1'; request ignored; loop continues	Pass	✓ Verified
TC-06	Floor 11	'Invalid floor: 11'; request ignored; loop continues	Pass	✓ Verified
TC-07	Floors 8, 2 (current: 0)	UP to 8, then DOWN to 2; direction changes correctly	Pass	✓ Verified
TC-08	Single floor 10	UP traversal 0→10 step-by-step; door open/close at 10	Pass	✓ Verified

5.2 Edge Cases Handled

- Floor equals current floor: IdleState is assigned; a status message is printed and the request completes without any movement.
- Floor below 0: Guard condition rejects immediately with a descriptive message; queue continues processing remaining valid requests.
- Floor above 10: Identical rejection path — system remains stable and continues.
- Mixed direction requests: State is re-evaluated per floor, so a sequence like [3, 7, 2] is handled correctly — up to 3, up to 7, then down to 2.
- Single request only: Functions correctly — one directional sweep with one arrival and door operation cycle.

5.3 Failure Scenarios

- Non-integer input: Scanner.nextInt() throws InputMismatchException. Since Main does not wrap the input loop in a try-catch, this would terminate the program — a known limitation identified for future improvement.
- Negative floor: Caught by the guard in requestFloor(); prints 'Invalid floor: [n]' and returns without enqueueing.
- Floor > 10: Same path as negative floor — graceful rejection with a printed message.

6. Git Workflow

6.1 Commit History

The screenshot below shows the full commit history on GitHub, reflecting 10 commits spread across 4 development days (March 17–25, 2026).

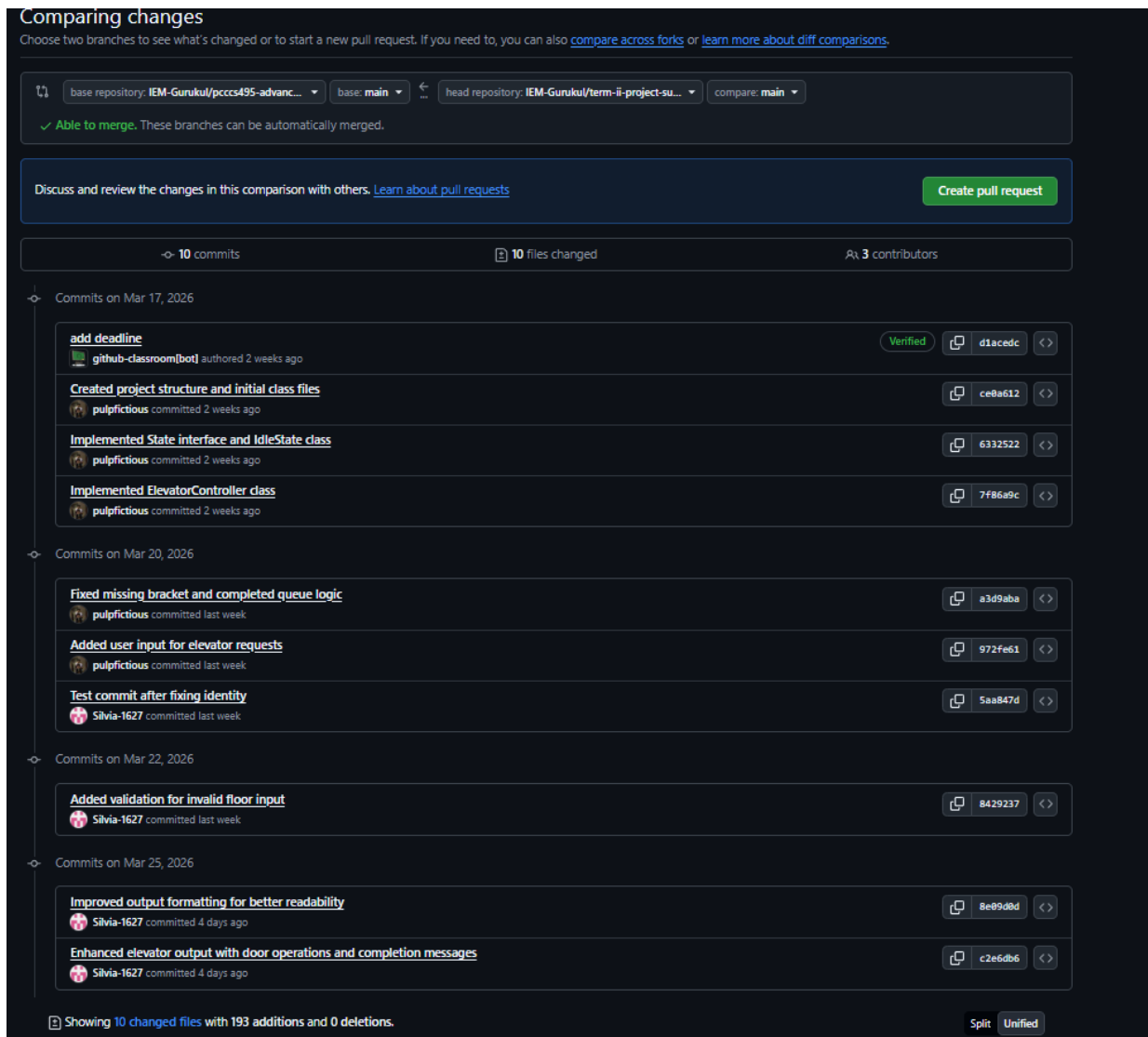


Figure 2: GitHub commit history — 10 commits across 4 days

6.2 Development Progression

Each commit represents a meaningful, self-contained step in the development lifecycle:

```
d1acedc | Mar 17 - add deadline (repository setup via GitHub Classroom)
ce0a612 | Mar 17 - Created project structure and initial class files
6332522 | Mar 17 - Implemented State interface and IdleState class
7f86a9c | Mar 17 - Implemented ElevatorController class
a3d9aba | Mar 20 - Fixed missing bracket and completed queue logic
972fe61 | Mar 20 - Added user input for elevator requests
5aa847d | Mar 20 - Test commit after fixing identity
8429237 | Mar 22 - Added validation for invalid floor input
8e09d0d | Mar 25 - Improved output formatting for better readability
c2e6db6 | Mar 25 - Enhanced elevator output with door operations and completion
messages
```

The commit history reflects disciplined, incremental development. The project began with structural setup and core interface design on March 17, progressed through logic implementation and bug fixing on March 20, added robustness through input validation on March 22, and concluded with output polish on March

25. This progression demonstrates thoughtful, iterative software development rather than a single monolithic submission.

7. Conclusion & Future Scope

7.1 Conclusion

This project successfully demonstrates the design and simulation of an elevator control system in Java, meeting all stated objectives. A LinkedList-backed Queue provides clean FIFO request management, and the State Design Pattern ensures each operational phase — idle, moving up, moving down — is encapsulated in a dedicated, focused class. The result is a system where adding a new state requires only a new class, with zero changes to existing logic.

The deliberate use of a Queue collection, step-by-step floor traversal, input validation, and interface-driven polymorphism all reflect considered design choices that improve robustness, readability, and extensibility. The project demonstrates how core OOP principles — abstraction, polymorphism, encapsulation, exception handling, and collections — work together cohesively in a real-world simulation context. Every design decision made in this project is defensible and grounded in software engineering best practice.

7.2 Future Scope

- **SCAN Scheduling:** Replace the FIFO Queue with a PriorityQueue to implement SCAN-style scheduling — serving floors in sorted directional order, reducing unnecessary back-and-forth movement.
- **Multi-elevator support:** Introduce a Dispatcher class managing multiple Elevator instances and routing each request to the nearest or least-loaded cabin.
- **Request object integration:** Wire the existing Request class (with sourceFloor and destinationFloor) into the controller for origin-aware dispatching.
- **Thread.sleep() for realism:** Add simulated travel delay between floors to make console output reflect real elevator timing.
- **Graphical interface:** Replace the CLI with a JavaFX GUI visualising elevator movement in real time across a building floor plan.
- **Input robustness:** Wrap Scanner.nextInt() in a try-catch for InputMismatchException to handle non-integer inputs gracefully without crashing the session.

Appendix

A. Initial Project Proposal

Project Title: Elevator Control System Simulation Using Object-Oriented Programming and State Design Pattern

1. Introduction / Background

Elevators play a critical role in modern multi-storey buildings such as offices, hospitals, shopping malls, and universities. Although using an elevator seems simple from a user's perspective, the system behind it must manage many requests from different floors efficiently. This project builds a simulation using OOP in Java, implementing the State Design Pattern to model different operational phases — idle, moving up, moving down, and door open.

2. Problem Statement

In multi-floor buildings, elevators receive many simultaneous requests. Without a proper control strategy, the elevator may move inefficiently, causing long wait times. The challenge is to design a system that receives requests, organises them, determines correct direction and movement, and manages distinct operational states.

3. Objectives

- Design and implement a simulation of an elevator control system.
- Apply OOP principles — abstraction, polymorphism, encapsulation, exception handling, and collections.
- Understand how system states affect program behavior using the State Design Pattern.
- Practice modular software design using multiple interacting classes.
- Implement data structures and exception handling in Java.

4. Core Features

- Accept floor requests from the user at runtime via Scanner.
- Validate floor inputs and reject out-of-range values gracefully.
- Queue and process multiple requests in FIFO sequence.
- Move the elevator floor-by-floor in the correct direction.
- Simulate door open and door close operations upon arrival at each floor.

5. OOP Concepts Used

The project will demonstrate: Encapsulation (Elevator fields private), Abstraction (State interface), Polymorphism (state-based dispatch), Exception Handling (invalid floor rejection), and Collections (Queue for request management).